

A ratio-closure simplex for maximum-ratio closure and its canonical decomposition: implementation and experimental report

Experimental report generated from the stored campaigns

September 6, 2026

Abstract

This report documents the measurement campaigns run on the ratio-closure simplex solver of this project and on the two independent combinatorial backends built as references. It states the questions asked, the instances and protocol used, the validation performed on every result, how the solver's parameters were tuned on data disjoint from the test set, and what the measurements do and do not support. Every table and figure is regenerated from the stored run manifests by `tools/report/analyze.py`. The figures quoted in the running text are taken from those same tables and from the stored manifests, and were re-checked against them after the instance library was regenerated deterministically and every campaign re-run (`docs/decisions.md`, D16).

Contents

Summary of findings	2
1 Objectives and hypotheses	3
2 Methods	4
2.1 Problem and required output	4
2.2 Compared methods	4
2.3 Accuracy is fixed before speed	5
3 Environment	6
4 Instances	6
4.1 Generated families	6
4.2 Historical PCKP instances	7
4.3 Open-pit instances derived from MineLib	7
4.4 Frozen splits	7
5 Validation	7
6 Tuning	8
6.1 Search space and protocol	8
6.2 Frozen configuration	9
7 Main results on the frozen test set	9
7.1 Against the 2023 prototype	10
8 The full canonical decomposition	12

Table 1: Headline of the frozen test set. *solved* counts the instances finished and independently verified within the limit; *median* and *worst* are wall-clock seconds over those; *PAR-2* sums the times and charges twice the time limit for each unsolved instance, so lower is better and a single time-out costs a lot; *MiB* is the largest peak resident memory observed.

configuration	solved	median	worst	PAR-2	MiB
dinkelbach	40/46	0.234	23.383	447	364
forest-reference	27/46	0.332	29.433	1242	320
forest-tuned	36/46	0.142	24.711	653	320
parametric	46/46	0.057	26.362	47	384

9	Where the time goes	14
9.1	What separates the easy families from the hard ones	17
9.2	The dual simplex	17
10	Scalability and many capacities	19
10.1	One capacity against many	19
11	Degeneracy, ties and numerics	21
11.1	Why the pivot loop stalls	21
11.2	Ties and numerics	21
12	Discussion	23
12.1	What the measurements support	23
12.2	What changed, and why it matters more than the parameters	23
12.3	What the measurements do not support	24
12.4	What follows	24
13	Reproducibility	24
A	Complete tables and campaign details	24

Summary of findings

What was measured. One linear program, three independent solvers. The ratio-closure simplex of this project, a Dinkelbach iteration whose ratio oracle is a minimum cut, and a divide-and-conquer parametric decomposition. All three return the same object — an optimal LP solution, its multipliers and the canonical sequence of blocks — and every result is re-checked by a separate process before it is counted.

Correctness. The three backends agree everywhere they were compared, and agree with an exhaustive rational oracle on small instances and with an independent LP solver on larger ones. Nothing in the campaigns contradicts the correctness of any backend.

Speed. The parametric backend is the only configuration that solves the whole test set: 46 of 46. Dinkelbach solves 40, the tuned ratio-closure simplex 36 and the reference ratio-closure simplex 27. Two families are solved by the parametric backend alone — disconnected graphs and the instances derived from MineLib — where the other three finish nothing within the limit.

The medians do not rescue the counts. Over the instances it solves, the tuned ratio-closure simplex has a median of 0.142s against the parametric backend’s 0.057s, and that median is taken over the smaller subset it can finish; Dinkelbach sits between them at 0.234s but solves four more instances. On the historical precedence-knapsack instances all four configurations

except the reference finish everything, and the ordering there is unambiguous: 0.027 s median for the parametric backend, 0.087 s for Dinkelbach, 0.655 s for the tuned ratio-closure simplex. The honest summary is that this implementation is neither the fastest where it finishes nor able to finish everywhere.

What made the difference. Not the pricing rule. The first measurements had ratio-closure simplex one to three orders of magnitude behind, and parameter search over pricing schemes moved the result by a few per cent. What moved it by factors was the handling of degeneracy: the primal spends a median of 99.3% of its pivots on zero steps, and three changes follow from that — releasing the anti-cycling fallback instead of letting it become the operating rule, restricting the ratio-test scan to the arcs that can actually block, and stopping that scan at the first zero-step blocker. The last two reproduce techniques already present in the group’s historical prototype.

The ablation confirms the ranking and corrects two of the search’s choices. Measured as paired speedups against the setting that forces Bland’s rule, the frozen configuration is 2.83 times faster at the median and up to 18.8 times on individual instances, so the anti-cycling release is worth a factor rather than a percentage; moving the degeneracy trigger back to its old value gives up most of that, 1.25 instead of 2.83. The ratio-test refinements are worth about a factor of two each, 2.17 and 2.20.

Two settings the search did *not* choose are faster than the one it did: starting from the sink basis scores 3.33 and enabling transitive reduction 3.46, against the frozen configuration’s 2.83. The frozen configuration is reported as it was frozen, on training and validation data only, and not silently improved after the fact — but the ablation says plainly that the search left something on the table, and Section 12 treats it as a finding rather than a footnote.

How to read the tables. Times are wall-clock medians over replicas of the same instance and configuration. *PAR-2* charges twice the time limit for every instance a configuration did not solve, so it mixes speed and robustness into one number; it is always reported next to the plain count of solved instances so that a time-out is never hidden inside an average. Paired speedups are computed only on the instances both settings solved and verified. Performance profiles state the compared set and never count an unsolved instance as covered.

What this report does not claim. It does not claim novelty: the audit of the literature was deliberately not carried out in this phase. It does not claim anything about the state of the art in maximum flow: the flow backends use a Dinic implementation written for this project, not a competitive flow code.

1 Objectives and hypotheses

The object of the evaluation is the ratio-closure simplex solver: a combinatorial simplex whose basis is represented by a forest on the precedence graph, applied to the normalised maximum-ratio subproblem and iterated on the residual to build the canonical sequence of blocks that yields the LP value at any capacity.

Two further backends were implemented for this report and are used as references, not as contributions: a Dinkelbach iteration whose ratio oracle is a maximum-closure minimum cut, and a divide-and-conquer parametric decomposition of the same closure family. All three produce the same certified structures and are checked by the same verifiers.

The campaigns answer the following questions.

E1 Correctness

Does the solver produce certified optimal solutions on generic DAGs, with negative profits, ties and degeneracy? Do the three backends and an independent LP solver agree?

E2 Overall performance

How do the reference ratio-closure simplex configuration, the tuned one and the two combinatorial baselines compare on a frozen test set, per family rather than only in aggregate?

E3 Scalability

How do time and memory grow with n and m , and how much of the growth is the cost of a pivot rather than the number of pivots?

E4 Ablation

What does each implementation choice contribute: local forest updates, dual canonicalisation, partial pricing, transitive reduction, vertex ordering?

E5 Tuning

Does the search produce a default that transfers from training and validation to instances never used to choose it?

E6 One capacity against many

When does building the whole positive path pay for itself against a single solve, once the cost of the queries and of materialising the solution is included?

E7 Degeneracy and numerics

What is the cost of ties, exact and near-exact, and of the anti-cycling fallback?

E8 Applied instances

What happens on the historical PCKP instances and on the open-pit instances derived from MineLib?

No question is posed so that a particular answer is the only admissible one. In particular, the report states plainly where the ratio-closure simplex implementation is slower than the baselines, and section 12 discusses why.

2 Methods

2.1 Problem and required output

All backends solve the same linear program: maximise $\sum_i p_i x_i$ subject to $\sum_i w_i x_i \leq c$, $x_i \leq x_j$ for every arc (i, j) meaning “ i requires j ”, and $0 \leq x_i \leq 1$. Profits carry either sign, weights are strictly positive, and there is a single capacity row. Cyclic digraphs are accepted and condensed; solutions and multipliers are expanded back to the original variables and arcs.

The required output of a `solve` task is a primal solution, the multipliers λ , μ and α , and the canonical blocks of the positive path. Comparisons are therefore between runs that produce the same object: whenever a configuration answers several capacities from one path, the report says so explicitly and section 10 treats it separately.

2.2 Compared methods

A note on the labels. The method developed by this project is the ratio-closure simplex, and the configuration labels in the current suites are named after it, `ratio-closure-*`. The labels below instead read `forest-*`, because that is what the campaigns reported here actually recorded: they were measured on 6 September 2026, before decision D18 was applied to the code, and a run identifier is derived from the instance, the task, the capacity, the configuration *content* and the hash of the sources. The manifests are a record of what was executed and are not rewritten, so this report quotes the labels as measured. The divergence is cosmetic and disappears with the first campaign measured under the current suites; the analysis script recognises both spellings.

Table 2: Measurement environment and build fingerprint recorded in the stored results.

item	value
CPU	Intel(R) Core(TM) i5-14500T
logical cores	20
memory	15 GiB
system	Ubuntu 26.04 LTS
compiler	GNU 15.2.0
build type	Release
build flags	-O3 -DNDEBUG;sanitizers=OFF;LTO=OFF;native=OFF
source hash	493f6f942eca3576...
distinct builds	2
solver affinity	one core per solver process (<code>taskset</code>)
concurrency	one campaign at a time, no concurrent measurement

forest-reference

ratio-closure simplex with full pricing, Bland’s entering rule, complete forest rebuilds and sequential canonicalisation. This is the verifiable reference implementation, not a configuration meant to be fast.

forest-tuned

the configuration frozen by the random search of section 6, chosen on training and validation data only.

dinkelbach

Newton iteration on $\max\{p(S) - \lambda w(S)\}$ over closures, started from a strict lower bound on the ratio so that negative ratios need no special case. Each iterate is a maximum-closure minimum cut.

parametric

divide and conquer on the interval of ratios: the mean ratio of an interval is either a breakpoint or splits it in two. Each subproblem is an independent maximum flow.

The maximum-flow routine is a Dinic implementation written for this project (level graphs, current arc, iterative augmentation). It is correct and reasonable, but it is *not* a state-of-the-art flow code: no pseudoflow, no highest-label push-relabel with gap and global relabelling, no reuse of flow across parametric subproblems. Any statement about the flow backends in this report is a statement about this implementation, and section 12 says what that does and does not license.

2.3 Accuracy is fixed before speed

All configurations use the same tolerances, and tolerances are not part of the tuning space. Every run must pass, in the original units, the primal checks ($0 \leq x \leq 1$, every original arc, the capacity row), the dual sign conditions, the reduced-cost inequality $w_i \lambda + \mu_i + \text{div}_i(\alpha) \geq p_i$, complementarity on μ , on α and on the capacity row, and equality between the primal objective and the dual bound. A result that fails any of these is reported as `numerical_failure` and never counted as a success.

Verification is a separate process: after each run, `pclp verify` reloads the instance and the saved result and re-checks the certificate without re-optimising. Only runs that pass this second, independent check enter the analysis.

Table 3: Instance families, sizes and how many instances of each family fall in the frozen training, validation and test manifests.

family	count	n	m	train	validation	test
MineLib (derived)	8	1060–99014	3922–1271207	4	2	2
PCKP (historical)	23	349–11757	1661–83218	12	6	5
SCCs	9	200–20000	209–20009	3	0	6
chains	18	200–20000	199–19999	9	6	3
dense DAGs	9	200–2000	1968–200313	3	3	3
diamonds	9	200–20000	244–24994	6	0	3
disconnected	15	200–20000	193–19993	9	3	3
layered DAGs	33	200–8000	213–350302	18	6	9
no arcs	18	200–20000	0–0	9	3	6
precedence forests	18	200–20000	191–19991	9	6	3
sparse DAGs	24	200–20000	577–63777	12	9	3
transitive arcs	9	200–8000	1564–63968	3	6	0
two levels	9	200–8000	190–320402	6	3	0
total	202			103	53	46

3 Environment

All measurements come from a single machine and a single build. The solver is compiled in **release** mode without link-time optimisation and without `-march=native`, so that the reported times refer to a portable build; sanitiser builds and per-pivot tracing exist but are never used for measurement. Each solver process is pinned to one core, and campaigns are run one at a time. This is not a precaution taken on principle: during the preliminary exploration the same configuration on the same instance measured about 40 seconds with a single active process and over 60 seconds with three processes pinned to distinct cores of the same machine. Pinning does not remove contention for shared cache and memory bandwidth, so concurrent campaigns were ruled out and the wall-clock cost was accepted instead.

The build fingerprint stored in every result contains the SHA-256 of all sources that enter the binary, the compiler, the build type and the flags. A run whose build hash differs from the campaign’s is not counted in that campaign: changing the code produces new run identifiers rather than silently mixing measurements.

Wall time is measured by the runner with a monotonic clock around the whole process, including reading the instance and writing the result. The solver separately reports the time of its own phases. Peak memory is the solver process’s `ru_maxrss`, kept distinct from the runner’s memory.

4 Instances

4.1 Generated families

Each synthetic family isolates one structural phenomenon: absence of arcs (overhead only), chains (maximum depth), precedence forests, layered DAGs with and without level-skipping arcs, sparse and dense random DAGs, diamonds and shared prerequisites, disconnected components, transitively redundant arcs, artificial strongly connected components, and two-level graphs. Across these, the generator varies the size, the weight spread (uniform, dispersed, heterogeneous by orders of magnitude), the share of negative profits, the correlation between profit and weight, and the share of exact ties. Every instance stores its generator, parameters and seed, so the whole set is rebuilt byte for byte by one command.

Table 4: Validation evidence: independent oracles used by the acceptance suite, and the outcome of every stored campaign run. A run counts as verified only if a separate `pclp verify` call re-checked its certificate.

check	coverage	outcome
exhaustive rational oracle	308 instances	4628 solver calls
basis algebra in exact arithmetic	66 pivots	4 pivot types
independent LP (HiGHS via SciPy)	0 instances	scipy
Dinkelbach against the ratio-closure sequence	80 instances	same blocks and LP values
parametric against the ratio-closure sequence	80 instances	same blocks and LP values
campaign ablation	560/620 verified	optimal 560, time_limit 60
campaign canonical_path	130/144 verified	optimal 130, time_limit 14
campaign capacities	372/372 verified	optimal 372
campaign dual	234/248 verified	optimal 234, time_limit 14
campaign main	447/552 verified	optimal 447, time_limit 105
campaign pilot	40/48 verified	optimal 40, time_limit 8
campaign scalability	518/612 verified	optimal 518, time_limit 94
campaign smoke	16/16 verified	optimal 16

4.2 Historical PCKP instances

Twenty-three single-capacity precedence-constrained knapsack instances were imported from the project’s historical archive, with 349 to 11 757 vertices and up to 83 218 arcs. Each was validated against the companion CPLEX `.lp` model: the importer accepts only one capacity row plus simple precedence rows with unit bounds, and refuses everything else. Because the two files do not always share the variable numbering, the check falls back to colour refinement and claims a correspondence only when every colour class is a singleton and the mapped arc sets coincide exactly. The multi-period files of the same archive have one capacity row per period and are therefore outside this family; they were refused, not adapted.

4.3 Open-pit instances derived from MineLib

Eight open-pit instances, up to 99 014 blocks and 1 271 207 precedence arcs, were derived from the MineLib CPIT models. The derivation is explicit and recorded inside every imported instance: period indices and discounting are dropped, the knapsack row is resource 0, and the capacity is that resource’s limit summed over the periods it was granted. These are realistic structural benchmarks, not the original MineLib models, and the report never presents them as such. Three MineLib instances were refused because some blocks have no resource-0 coefficient and would carry zero weight, outside the supported family.

4.4 Frozen splits

Training, validation and test manifests were written once, stratified by origin, family and size decade, keeping all replicas of one generated base on the same side. Parameter search uses training only, the finalists are compared on validation, the chosen configuration is frozen with its date and rationale, and only then is the test set measured.

5 Validation

Correctness is established at three levels, each with a calculation independent of the solver being checked.

Exhaustive rational oracle. For small instances the acceptance suite enumerates every subset, keeps the closures, and computes with exact fractions the maximum ratio, *all* its maximisers and their canonical union, then repeats on the residual to obtain the whole sequence. The LP value is obtained independently from the convex hull of the points $(w(C), p(C))$, including pairs of closures straddling the capacity, without assuming that the two are nested. The solver must reproduce the same blocks, the same ratios and the same LP values.

Basis algebra in exact arithmetic. For traced runs the suite rebuilds, in exact arithmetic, the explicit basis matrix of every pivot and recomputes the basic values, the direction, the reduced cost, the ratio-test step, the leaving variable and the pivot coefficient. It requires that all four structural pivot types — vertex or slack entering against vertex or slack leaving — actually occur.

Independent LP solver. The same instances are handed to HiGHS through SciPy as a plain LP with one variable per vertex and one row per arc. Only the optimal value is compared: the optimal vector need not be unique.

Cross-backend agreement. The Dinkelbach and parametric backends produce their own certificates from their own flows, and are required to return the same canonical blocks and the same LP values as ratio-closure simplex. Because they share no code with the pivot loop, an error in the forest engine cannot be masked by an identical error in its verifier.

Metamorphic checks. Permuting vertex identifiers and arc order, duplicating precedence rows, adding transitively redundant arcs, adding an isolated negative vertex, scaling profits, and scaling weights and capacity together all leave the value and the canonical blocks where they should be. Condensing an artificial strongly connected component must agree with the equivalent aggregated vertex.

Per-run verification. Beyond the suite, every run of every campaign is re-checked by a separate `pclp verify` process that reloads the instance and the saved result and validates the certificate without re-optimising. The analysis counts only runs that pass this second check.

6 Tuning

6.1 Search space and protocol

Only parameters the solver actually implements take part in the search: pricing scheme and its block size or candidate limit, entering rule, forest update mode with its rebuild interval and fraction, canonicalisation, degeneracy trigger, vertex ordering of the condensation, transitive reduction and the initial basis. Conditional parameters are sampled only when their guard holds, so a candidate never carries a block size it cannot use, and the constraint that Bland’s rule requires full pricing mirrors the solver’s own validation. Numerical tolerances are deliberately excluded: relaxing them would buy time by solving a different problem.

Candidates are drawn by reproducible random search from a fixed seed, after a small list of a-priori plausible configurations. Each candidate is evaluated on a deterministic subsample of the training manifest under a fixed per-run limit, and ranked lexicographically by number of incorrect results, number of instances solved and verified within the limits, PAR-2 time, and peak memory. Every attempt, including rejected ones and the reason for rejection, is appended to a history file, so an interrupted search resumes without repeating work and the total cost of tuning — not only the cost of the winning configuration — is recorded.

Table 5: Random search on data/splits/train.json: best 8 of 12 attempts, total tuning time 560 s.

attempt	configuration	solved	failures	PAR-2
cfg002_4ba55896	candidate_limit=64, canonicalization=dual, degeneracy_trigger=32768, entering_rule=best-improving, forest_upda	11/16	0	67.7
cfg005_23d37fdc	canonicalization=sequential, degeneracy_trigger=1024, dual_leaving=first, entering_rule=bland, forest_update=a	11/16	0	69.5
cfg004_ca61ba7e	canonicalization=sequential, degeneracy_trigger=32768, dual_leaving=first, entering_rule=first-improving, fore	11/16	0	72.0
cfg003_d35a4751	canonicalization=dual, degeneracy_trigger=32768, entering_rule=best-improving, forest_update=adaptive, initial	10/16	0	78.3
cfg009_d288e581	canonicalization=sequential, degeneracy_trigger=128, dual_leaving=first, entering_rule=best-improving, forest_	10/16	0	79.3
cfg000_f9525dc0	canonicalization=dual, degeneracy_trigger=32, entering_rule=bland, forest_update=local, initial_basis=sink, no	10/16	0	80.6
cfg010_dba2c526	canonicalization=sequential, degeneracy_trigger=32768, entering_rule=first-improving, forest_update=adaptive,	10/16	0	81.6
cfg001_799706a1	canonicalization=dual, degeneracy_trigger=32768, entering_rule=best-improving, forest_update=local, initial_ba	10/16	0	85.6

A first search was run before the parameters that turn out to matter existed in the solver, and its attempts differed by less than three per cent: it is recorded in `experiments/tuning/` but it selected nothing and is not reported here. The search shown above is the one whose winner was frozen.

6.2 Frozen configuration

The configuration selected on training and validation data is written to `configs/frozen/` together with its date, the split it was chosen on, the search space and the rationale, and only then is the test manifest measured. The test set is never used to adjust a default.

7 Main results on the frozen test set

Table 6: Campaign main: instances solved and verified, median and worst median wall time in seconds, PAR-2 with a 60 s limit, and peak resident memory in MiB.

family	configuration	solved	median	worst	PAR-2	MiB
MineLib (derived)	dinkelbach	1/2	0.168	0.168	60.08	364
MineLib (derived)	forest-reference	1/2	5.889	5.889	62.94	320
MineLib (derived)	forest-tuned	1/2	1.109	1.109	60.55	320
MineLib (derived)	parametric	2/2	13.192	26.362	13.19	384
PCKP (historical)	dinkelbach	5/5	0.087	18.265	0.09	34
PCKP (historical)	forest-reference	3/5	5.326	10.967	10.97	34
PCKP (historical)	forest-tuned	5/5	0.655	24.711	0.66	34
PCKP (historical)	parametric	5/5	0.027	0.690	0.03	34
SCCs	dinkelbach	6/6	0.006	0.008	0.01	27
SCCs	forest-reference	6/6	0.006	0.010	0.01	27
SCCs	forest-tuned	6/6	0.006	0.008	0.01	27
SCCs	parametric	6/6	0.006	0.009	0.01	27
chains	dinkelbach	3/3	0.007	0.007	0.01	22
chains	forest-reference	3/3	0.007	0.007	0.01	22
chains	forest-tuned	3/3	0.005	0.006	0.00	22
chains	parametric	3/3	0.005	0.005	0.00	22
dense DAGs	dinkelbach	3/3	0.674	0.684	0.67	27
dense DAGs	forest-reference	0/3	–	–	120.00	27
dense DAGs	forest-tuned	3/3	0.746	0.751	0.75	27
dense DAGs	parametric	3/3	0.105	0.122	0.10	27
diamonds	dinkelbach	3/3	0.237	0.315	0.24	27
diamonds	forest-reference	3/3	0.875	1.328	0.87	27
diamonds	forest-tuned	3/3	0.110	0.161	0.11	27
diamonds	parametric	3/3	0.041	0.054	0.04	27
disconnected	dinkelbach	0/3	–	–	120.00	35
disconnected	forest-reference	0/3	–	–	120.00	35

continua

Table 7: Campaign main: paired speedup against forest-reference on the instances both settings solved and verified.

configuration	paired	median	min	max
dinkelbach	27	1.45	0.54	184.71
forest-reference	27	1.00	1.00	1.00
forest-tuned	27	3.49	0.51	48.78
parametric	27	9.54	0.83	430.70

continua dalla pagina precedente

family	configuration	solved	median	worst	PAR-2	MiB
disconnected	forest-tuned	0/3	–	–	120.00	35
disconnected	parametric	3/3	2.804	4.231	2.80	35
layered DAGs	dinkelbach	8/9	1.926	23.383	1.93	34
layered DAGs	forest-reference	3/9	0.011	0.013	120.00	34
layered DAGs	forest-tuned	3/9	0.018	0.021	120.00	34
layered DAGs	parametric	9/9	0.089	0.771	0.09	34
no arcs	dinkelbach	5/6	0.688	3.036	1.83	39
no arcs	forest-reference	4/6	0.353	29.433	14.90	39
no arcs	forest-tuned	6/6	0.324	8.496	0.32	39
no arcs	parametric	6/6	0.227	6.095	0.23	39
precedence forests	dinkelbach	3/3	1.568	1.577	1.57	27
precedence forests	forest-reference	3/3	6.239	8.263	6.24	27
precedence forests	forest-tuned	3/3	1.171	1.787	1.17	27
precedence forests	parametric	3/3	0.053	0.066	0.05	27
sparse DAGs	dinkelbach	3/3	0.782	1.092	0.78	27
sparse DAGs	forest-reference	1/3	25.855	25.855	120.00	27
sparse DAGs	forest-tuned	3/3	1.873	2.121	1.87	27
sparse DAGs	parametric	3/3	0.064	0.068	0.06	27

The comparison is between runs that produce the same object: a certified primal solution, the multipliers, and the canonical blocks of the positive path at the instance’s own capacity. Instances that no configuration solved within the limit are still counted in PAR-2 and in the solved column; they are not removed from the sample.

7.1 Against the 2023 prototype

The group’s 2023 ratio-closure simplex prototype was imported into this project under `legacy/forest2023/` and measured on the same maximum-ratio oracle, which is the task it implements. Three patches were needed to run it at all and are recorded in that directory: an absolute CPLEX include replaced by opaque typedefs, four interactive `cin.get()` pauses removed, and the append to a file in the working directory removed. No line of the algorithm was changed.

Running it exposed limits that belong to the prototype, not to the comparison. It stores profits and weights as `int`, so instances with fractional data cannot be represented and are refused instead of rounded. It allocates two $n \times n$ adjacency matrices, so its memory is quadratic

Table 8: Agreement of the LP value with dinkelbach on the instances both configurations solved and verified: worst relative difference and number of pairs above 10^{-9} .

configuration	paired	worst relative gap	above 10^{-9}
forest-reference	27	0.00e+00	0
forest-tuned	35	0.00e+00	0
parametric	40	0.00e+00	0

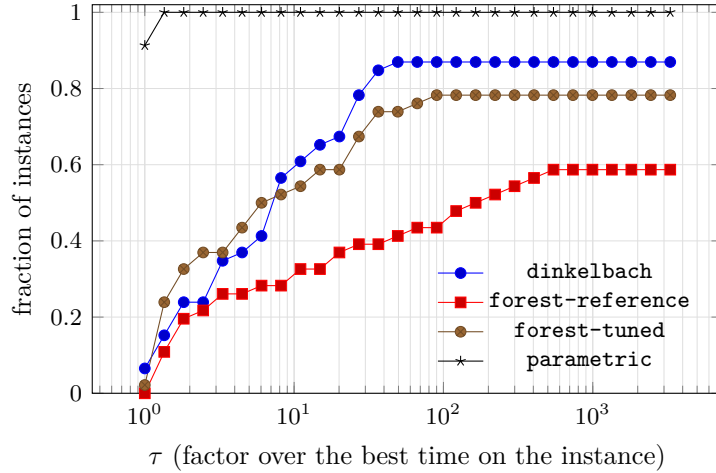


Figure 1: Performance profile on the frozen test set. A configuration is counted at τ when its median time on an instance is within a factor τ of the best median time recorded on that instance; unsolved instances never enter the numerator. The compared set is the instances at least one configuration solved and verified.

in the number of vertices. Its initial basis assumes a spanning tree of $n - 1$ edges, so it requires a connected graph. Finally, the feasibility shortcut added in the 2023-05-11 version recurses without termination on a generic, non-bipartite DAG — it overflows the stack already on the eight-node example — so the measurements use it disabled, which reproduces the behaviour of the 2023-05-05 version of the same file.

The outcome. On the eighteen instances measured, the prototype completed *none*, so there is no time to compare it on and no table worth printing. Four times it hit the time limit — including a chain of two hundred vertices, which the current solver certifies in nine milliseconds — once it crashed, once it exceeded the memory budget, and the remaining twelve it refused outright, either because the data are not integral or because the graph is disconnected and no spanning tree of $n - 1$ edges exists. The current solver returned a certified optimal ratio on every one of the eighteen, in times between seven milliseconds and thirty-two seconds. The per-instance record is in `experiments/analysis/legacy\_baseline*.json`.

That is a wide gap, and it should be read for what it is. Most of it is not a difference in pivot rules: it is that the prototype rebuilds the whole forest and reallocates two arrays at every iteration, recurses instead of iterating, and carries data structures quadratic in the number of vertices. Those are the costs of a research prototype that was never asked to run outside the bipartite instances it was written for, and the current solver inherits none of them. The comparison therefore measures engineering, not the merit of the underlying method — which is the same method in both.

What the prototype did contribute is two ideas that survived measurement and are in the current solver: the early exit of the ratio test on a zero-step blocker, which it implemented as an explicit feasibility check during the forest traversal, and the absence of a permanently engaged anti-cycling rule, which the ablation of section 9 confirms was costing a factor in pivots. A third idea is still on the table and not yet implemented: the Euler-tour intervals it maintained, which give subtree membership in constant time and would attack the rebuild cost that now dominates.

Where both solvers return an optimal ratio, they return the same one. On this instance set that condition never held, so the cross-check rests on the eight-node example of the tutorial, where both return the ratio 2.

8 The full canonical decomposition

Every campaign in Section 7 answers a single capacity: the solver walks the canonical sequence only as far as the capacity requires. This section asks the other question — produce the *entire* sequence of closure layers, task **full-path** — because that is the object the method is designed to deliver, and because it is the setting in which a basis, unlike a flow state, could in principle be carried from one residual to the next.

The suite is `configs/suites/canonical\_path.json`: twelve instances across the generated families, one historical PCKP instance and one MineLib instance, two replicas, a sixty-second limit.

The same canonical sequence, restricted to precedence graphs that are directed forests, is the subject of the companion work (Dose et al., 2026), whose aggregation algorithms reach $\mathcal{O}(n \log n)$ and whose implementation is public at <https://github.com/fabiofurini/parametric-closure-forests>. What this report calls a canonical block is a *closure layer* there, and the per-vertex bound computed here is their *threshold* of a vertex. The comparison below is on general DAGs, where those algorithms do not apply.

Table 9: Campaign `canonical_path`: instances solved and verified, median and worst median wall time in seconds, PAR-2 with a 60 s limit, and peak resident memory in MiB.

family	configuration	solved	median	worst	PAR-2	MiB
MineLib (derived)	dinkelbach	1/1	0.293	0.293	0.29	23
MineLib (derived)	parametric	1/1	0.034	0.034	0.03	23
MineLib (derived)	ratio-closure-dual	1/1	1.420	1.420	1.42	23
MineLib (derived)	ratio-closure-tuned	1/1	1.776	1.776	1.78	23
MineLib (derived)	ratio-closure-warm	1/1	0.247	0.247	0.25	23
MineLib (derived)	ratio-closure-warm-dual	1/1	0.308	0.308	0.31	23
PCKP (historical)	dinkelbach	2/2	0.085	0.160	0.09	23
PCKP (historical)	parametric	2/2	0.019	0.029	0.02	23
PCKP (historical)	ratio-closure-dual	2/2	0.146	0.244	0.15	23
PCKP (historical)	ratio-closure-tuned	2/2	0.084	0.119	0.08	23
PCKP (historical)	ratio-closure-warm	2/2	0.047	0.048	0.05	23
PCKP (historical)	ratio-closure-warm-dual	2/2	0.047	0.050	0.05	23
SCCs	dinkelbach	1/1	0.011	0.011	0.01	23
SCCs	parametric	1/1	0.012	0.012	0.01	23
SCCs	ratio-closure-dual	1/1	0.012	0.012	0.01	23
SCCs	ratio-closure-tuned	1/1	0.012	0.012	0.01	23
SCCs	ratio-closure-warm	1/1	0.014	0.014	0.01	23
SCCs	ratio-closure-warm-dual	1/1	0.011	0.011	0.01	23
chains	dinkelbach	1/1	0.423	0.423	0.42	23
chains	parametric	1/1	0.095	0.095	0.10	23
chains	ratio-closure-dual	1/1	0.392	0.392	0.39	23
chains	ratio-closure-tuned	1/1	0.133	0.133	0.13	23
chains	ratio-closure-warm	1/1	0.079	0.079	0.08	23
chains	ratio-closure-warm-dual	1/1	0.066	0.066	0.07	23
dense DAGs	dinkelbach	1/1	1.138	1.138	1.14	23
dense DAGs	parametric	1/1	0.196	0.196	0.20	23
dense DAGs	ratio-closure-dual	1/1	3.536	3.536	3.54	23
dense DAGs	ratio-closure-tuned	1/1	1.138	1.138	1.14	23
dense DAGs	ratio-closure-warm	1/1	0.949	0.949	0.95	23
dense DAGs	ratio-closure-warm-dual	1/1	3.613	3.613	3.61	23
disconnected	dinkelbach	1/1	1.906	1.906	1.91	23
disconnected	parametric	1/1	0.183	0.183	0.18	23
disconnected	ratio-closure-dual	1/1	2.940	2.940	2.94	23

continua

Table 10: Campaign canonical_path: paired speedup against dinkelbach on the instances both settings solved and verified.

configuration	paired	median	min	max
dinkelbach	11	1.00	1.00	1.00
parametric	11	7.64	0.86	27.21
ratio-closure-dual	11	0.65	0.08	6.80
ratio-closure-tuned	10	0.90	0.17	4.10
ratio-closure-warm	10	2.12	0.22	12.53
ratio-closure-warm-dual	11	1.14	0.09	15.68

continua dalla pagina precedente

family	configuration	solved	median	worst	PAR-2	MiB
disconnected	ratio-closure-tuned	1/1	2.140	2.140	2.14	23
disconnected	ratio-closure-warm	1/1	0.152	0.152	0.15	23
disconnected	ratio-closure-warm-dual	1/1	0.122	0.122	0.12	23
layered DAGs	dinkelbach	1/1	2.011	2.011	2.01	24
layered DAGs	parametric	1/1	0.074	0.074	0.07	24
layered DAGs	ratio-closure-dual	1/1	23.957	23.957	23.96	23
layered DAGs	ratio-closure-tuned	0/1	—	—	120.00	23
layered DAGs	ratio-closure-warm	0/1	—	—	120.00	23
layered DAGs	ratio-closure-warm-dual	1/1	22.901	22.901	22.90	23
no arcs	dinkelbach	1/1	0.566	0.566	0.57	23
no arcs	parametric	1/1	0.074	0.074	0.07	23
no arcs	ratio-closure-dual	1/1	0.083	0.083	0.08	23
no arcs	ratio-closure-tuned	1/1	0.138	0.138	0.14	23
no arcs	ratio-closure-warm	1/1	0.128	0.128	0.13	23
no arcs	ratio-closure-warm-dual	1/1	0.139	0.139	0.14	23
sparse DAGs	dinkelbach	1/2	0.608	0.608	60.30	31
sparse DAGs	parametric	2/2	1.882	3.712	1.88	33
sparse DAGs	ratio-closure-dual	1/2	0.824	0.824	60.41	28
sparse DAGs	ratio-closure-tuned	1/2	0.754	0.754	60.38	28
sparse DAGs	ratio-closure-warm	1/2	0.364	0.364	60.18	29
sparse DAGs	ratio-closure-warm-dual	1/2	0.283	0.283	60.14	29
transitive arcs	dinkelbach	1/1	1.309	1.309	1.31	24
transitive arcs	parametric	1/1	0.098	0.098	0.10	24
transitive arcs	ratio-closure-dual	1/1	3.032	3.032	3.03	24
transitive arcs	ratio-closure-tuned	1/1	1.591	1.591	1.59	24
transitive arcs	ratio-closure-warm	1/1	0.508	0.508	0.51	24
transitive arcs	ratio-closure-warm-dual	1/1	1.149	1.149	1.15	24

Restarting each oracle cold. With every residual oracle rebuilt from the empty basis — configurations **ratio-closure-tuned** and **ratio-closure-dual** — the method is not competitive. The parametric backend is faster on every instance but the arc-free graph, the median ratio is a factor of twelve, and on a layered DAG with $n = 2000$ it is a factor of 859. The oracle count is exactly what the design predicts, one per closure layer, and the parametric backend uses about two flow computations per layer. The difference is inside the oracle: each one costs tens to hundreds of thousands of pivots, of which between 98% and 100% are degenerate.

Restarting each oracle from the previous final basis. The extracted block *is* the positive component, that is one tree of the basis forest; the other trees survive untouched and stay anchored at zero. Reusing that forest is what **-warm-start residual** does, and the correctness condition is a single test: since the values are $1/w(T)$ on the positive tree T and zero elsewhere,

Table 11: Campaign main: share of the solver’s own measured time per phase, summed over the verified runs of each configuration.

configuration	runs	prep.	pricing	direction	ratio	update	certif.
dinkelbach	120	0.0	0.0	0.0	0.0	0.0	0.0
forest-reference	81	0.0	19.5	4.7	16.3	54.4	0.0
forest-tuned	108	0.1	6.7	14.5	12.4	48.1	0.0
parametric	138	0.4	0.0	0.0	0.0	0.0	0.1

Percentages of the solver’s internal accounting; the flow backends spend their time outside these ratio-closure phases and therefore show near-zero shares.

an active arc leaving T would violate feasibility, so T must be closed in the residual. The best closed surviving tree is chosen; if none is closed the oracle falls back to a cold restart. The check costs $\mathcal{O}(m)$ per oracle and is verified at every pivot in the acceptance suite.

The effect is on the pivot count, which is where the time was:

instance	cold pivots	warm pivots	factor
gen_disconnected_components6_n2000	251,780	3,080	81.7×
gen_sparse_degree3_n2000	89,468	6,108	14.6×
gen_transitive_degree2-extra3.0_n2000	166,730	13,225	12.6×
pckp_A_972_1661	8,503	1,056	8.1×
minelib_newman1	48,974	6,915	7.1×

The decomposition produced is identical block by block, and the objective of `solve` is unchanged to nine decimals.

The ranking, with the warm restart in. On the ten instances every configuration solves within the limit, the warm primal is 3.1 times faster than the cold one in total time and overtakes Dinkelbach, which the cold configuration did not. The simplex now wins three of the ten instances — the chain and the disconnected families, where the sequence has many layers, and the cyclic one — against one out of twelve before. The parametric backend still wins the remaining seven and is 3.4 times faster in total, so the conclusion of this report is unchanged in direction and changed in size: the gap is no longer one to two orders of magnitude but a factor of three, and it is closed entirely on the families with the longest sequences.

What this does and does not settle. It settles that the cold sequential decomposition was leaving most of its work on the table, and that the structure of the method — a basis, not a flow — is what recovers it. It does not settle the comparison against parametric flow, which still wins on the majority of instances and in total time. Two things are now worth measuring and are not measured here: the frozen configuration was tuned on `solve` and never on `full-path`, and a paired ablation on this task puts `initial_basis=sink` 1.21 times ahead of it; and the degeneracy that dominated the cold oracles is reduced, not removed, so the anti-cycling regime deserves its own tuning on this task.

9 Where the time goes

Table 12: Campaign ablation: instances solved and verified, median and worst median wall time in seconds, PAR-2 with a 60s limit, and peak resident memory in MiB.

family	configuration	solved	median	worst	PAR-2	MiB
PCKP (historical)	bland-entering	1/1	0.388	0.388	0.39	23
PCKP (historical)	close-degeneracy-trigger	1/1	0.388	0.388	0.39	23
PCKP (historical)	forest-tuned	1/1	0.041	0.041	0.04	23
PCKP (historical)	full-pricing	1/1	0.032	0.032	0.03	23
PCKP (historical)	no-early-ratio-test	1/1	0.052	0.052	0.05	23
PCKP (historical)	no-local-update	1/1	0.044	0.044	0.04	23
PCKP (historical)	no-restricted-ratio-test	1/1	0.058	0.058	0.06	23
PCKP (historical)	sequential-canonicalization	1/1	0.042	0.042	0.04	23
PCKP (historical)	sink-initial-basis	1/1	0.085	0.085	0.08	23
PCKP (historical)	with-transitive-reduction	1/1	0.045	0.045	0.05	23
chains	bland-entering	3/3	0.847	0.884	0.85	26
chains	close-degeneracy-trigger	3/3	0.506	0.553	0.51	26
chains	forest-tuned	3/3	0.076	0.096	0.08	26
chains	full-pricing	3/3	0.084	0.088	0.08	26
chains	no-early-ratio-test	3/3	0.098	0.118	0.10	26
chains	no-local-update	3/3	0.199	0.210	0.20	26
chains	no-restricted-ratio-test	3/3	0.092	0.112	0.09	26
chains	sequential-canonicalization	3/3	0.077	0.098	0.08	26
chains	sink-initial-basis	3/3	0.068	0.072	0.07	26
chains	with-transitive-reduction	3/3	0.078	0.094	0.08	26
dense DAGs	bland-entering	0/3	–	–	120.00	53
dense DAGs	close-degeneracy-trigger	0/3	–	–	120.00	53
dense DAGs	forest-tuned	3/3	10.733	14.776	10.73	57
dense DAGs	full-pricing	3/3	8.931	9.117	8.93	57
dense DAGs	no-early-ratio-test	0/3	–	–	120.00	53
dense DAGs	no-local-update	3/3	13.786	14.626	13.79	57
dense DAGs	no-restricted-ratio-test	0/3	–	–	120.00	53
dense DAGs	sequential-canonicalization	3/3	10.636	13.446	10.64	57
dense DAGs	sink-initial-basis	2/3	11.367	17.598	17.60	57
dense DAGs	with-transitive-reduction	3/3	1.665	1.920	1.67	43
disconnected	bland-entering	3/3	4.160	4.436	4.16	43
disconnected	close-degeneracy-trigger	3/3	3.803	4.102	3.80	43
disconnected	forest-tuned	3/3	1.130	1.249	1.13	43
disconnected	full-pricing	3/3	3.687	3.862	3.69	43
disconnected	no-early-ratio-test	3/3	1.820	1.879	1.82	43
disconnected	no-local-update	3/3	8.258	9.474	8.26	43
disconnected	no-restricted-ratio-test	3/3	1.722	1.864	1.72	43
disconnected	sequential-canonicalization	3/3	1.481	1.515	1.48	43
disconnected	sink-initial-basis	3/3	0.351	0.785	0.35	43
disconnected	with-transitive-reduction	3/3	1.127	1.283	1.13	43
layered DAGs	bland-entering	0/3	–	–	120.00	43
layered DAGs	close-degeneracy-trigger	0/3	–	–	120.00	43
layered DAGs	forest-tuned	3/3	5.976	6.726	5.98	43
layered DAGs	full-pricing	3/3	11.892	14.505	11.89	43
layered DAGs	no-early-ratio-test	3/3	16.102	19.986	16.10	43
layered DAGs	no-local-update	3/3	16.855	20.215	16.85	43
layered DAGs	no-restricted-ratio-test	3/3	14.851	18.493	14.85	43
layered DAGs	sequential-canonicalization	3/3	6.665	7.378	6.67	43
layered DAGs	sink-initial-basis	3/3	6.147	6.344	6.15	43
layered DAGs	with-transitive-reduction	3/3	6.435	6.821	6.43	43
no arcs	bland-entering	3/3	0.005	0.005	0.00	22

continua

continua dalla pagina precedente

family	configuration	solved	median	worst	PAR-2	MiB
no arcs	close-degeneracy-trigger	3/3	0.005	0.006	0.00	22
no arcs	forest-tuned	3/3	0.005	0.006	0.00	22
no arcs	full-pricing	3/3	0.005	0.005	0.00	22
no arcs	no-early-ratio-test	3/3	0.005	0.007	0.01	22
no arcs	no-local-update	3/3	0.005	0.005	0.00	22
no arcs	no-restricted-ratio-test	3/3	0.005	0.005	0.01	22
no arcs	sequential-canonicalization	3/3	0.005	0.005	0.00	23
no arcs	sink-initial-basis	3/3	0.005	0.005	0.00	23
no arcs	with-transitive-reduction	3/3	0.004	0.005	0.00	23
precedence forests	bland-entering	3/3	0.014	0.014	0.01	23
precedence forests	close-degeneracy-trigger	3/3	0.010	0.010	0.01	23
precedence forests	forest-tuned	3/3	0.008	0.011	0.01	23
precedence forests	full-pricing	3/3	0.010	0.012	0.01	23
precedence forests	no-early-ratio-test	3/3	0.009	0.010	0.01	23
precedence forests	no-local-update	3/3	0.017	0.020	0.02	23
precedence forests	no-restricted-ratio-test	3/3	0.008	0.010	0.01	23
precedence forests	sequential-canonicalization	3/3	0.008	0.010	0.01	23
precedence forests	sink-initial-basis	3/3	0.006	0.008	0.01	23
precedence forests	with-transitive-reduction	3/3	0.009	0.009	0.01	23
sparse DAGs	bland-entering	6/6	4.647	19.455	4.65	43
sparse DAGs	close-degeneracy-trigger	6/6	5.838	17.006	5.84	43
sparse DAGs	forest-tuned	6/6	0.525	1.316	0.52	43
sparse DAGs	full-pricing	6/6	0.784	2.534	0.78	43
sparse DAGs	no-early-ratio-test	6/6	0.743	2.892	0.74	43
sparse DAGs	no-local-update	6/6	1.600	4.503	1.60	43
sparse DAGs	no-restricted-ratio-test	6/6	0.717	2.515	0.72	43
sparse DAGs	sequential-canonicalization	6/6	0.517	1.311	0.52	43
sparse DAGs	sink-initial-basis	6/6	0.242	1.126	0.24	43
sparse DAGs	with-transitive-reduction	6/6	0.410	1.190	0.41	43
transitive arcs	bland-entering	3/3	0.034	0.037	0.03	23
transitive arcs	close-degeneracy-trigger	3/3	0.018	0.018	0.02	23
transitive arcs	forest-tuned	3/3	0.013	0.016	0.01	23
transitive arcs	full-pricing	3/3	0.015	0.016	0.01	23
transitive arcs	no-early-ratio-test	3/3	0.019	0.022	0.02	23
transitive arcs	no-local-update	3/3	0.017	0.019	0.02	23
transitive arcs	no-restricted-ratio-test	3/3	0.020	0.023	0.02	23
transitive arcs	sequential-canonicalization	3/3	0.014	0.015	0.01	23
transitive arcs	sink-initial-basis	3/3	0.011	0.012	0.01	23
transitive arcs	with-transitive-reduction	3/3	0.010	0.010	0.01	23
two levels	bland-entering	0/3	—	—	120.00	26
two levels	close-degeneracy-trigger	0/3	—	—	120.00	26
two levels	forest-tuned	3/3	7.009	11.055	7.01	26
two levels	full-pricing	0/3	—	—	120.00	26
two levels	no-early-ratio-test	3/3	19.922	25.801	19.92	26
two levels	no-local-update	2/3	17.295	17.896	17.90	26
two levels	no-restricted-ratio-test	2/3	26.566	26.772	26.77	26
two levels	sequential-canonicalization	3/3	10.826	11.167	10.83	26
two levels	sink-initial-basis	3/3	3.922	4.287	3.92	26
two levels	with-transitive-reduction	3/3	6.864	11.009	6.86	26

The breakdown separates the cost of one pivot from the number of pivots. The first is bounded by the structures: full pricing scans the $n - 1$ non-basic variables of the residual, the direction visits the entering subtree and the positive component, and the ratio test examines every basic slack of the residual, so a pivot costs $O(n + m)$ regardless of how small the change to the forest is. The second is a property of the instance and of the entering rule, and it is the

Table 13: Campaign ablation: paired speedup against bland-entering on the instances both settings solved and verified.

configuration	paired	median	min	max
bland-entering	22	1.00	1.00	1.00
close-degeneracy-trigger	22	1.25	0.66	2.50
forest-tuned	22	2.83	0.70	18.82
full-pricing	22	2.17	0.97	12.52
no-early-ratio-test	22	2.20	0.68	13.23
no-local-update	22	1.80	0.47	8.77
no-restricted-ratio-test	22	2.17	0.84	13.71
sequential-canonicalization	22	2.49	1.00	19.07
sink-initial-basis	22	3.33	0.98	41.71
with-transitive-reduction	22	3.46	1.01	24.21

quantity the ablation is designed to expose.

The ablation switches off one implementation choice at a time with respect to the frozen configuration, on the validation manifest, with the same data and the same acceptance thresholds as the main campaign.

9.1 What separates the easy families from the hard ones

It is tempting to attribute the cost to the number of blocks, since each one costs a fresh oracle with a fresh basis. The measurements do not support that. Across generated instances of two thousand vertices, all with the frozen configuration: a chain produces 11 blocks and 4640 pivots in 0.116 s; a sparse DAG of out-degree three produces 282 blocks and 134 830 pivots in 1.311 s; a layered DAG of twenty levels produces 232 blocks and 348 205 pivots in 6.079 s; and a layered DAG of eight levels with dense connections between them produces only 85 blocks — fewer than either sparse instance — yet 528 841 pivots, and does not finish within the limit. More blocks is not more work.

The quantity that separates them is pivots *per oracle*, expressed as a multiple of n : about $0.21n$ on the chain, $0.24n$ on the sparse DAG, $0.75n$ on the twenty-level instance and $3.1n$ on the eight-level dense one. The layered structure gives every entering variable many precedence slacks between vertices that are all at zero, so each oracle spends thousands of zero-step pivots walking to its optimum.

Two consequences follow for where effort should go. Reusing the basis across residual calls, which would save one initialization per oracle, cannot help the hard case: a hundred initializations against six hundred thousand pivots. What can help is either making each pivot cheaper — incremental pricing and updates confined to the modified paths, since pricing and rebuilding are together four fifths of the solver’s time on this family — or attacking the number of degenerate pivots directly with a lexicographic or perturbed ratio test. Only the second changes the multiplier, and it is the one with no implementation here.

This also explains the shape of the comparison with the parametric backend. That backend pays nothing for degeneracy: it solves one maximum flow per subproblem on disjoint vertex sets. Its advantage is largest exactly on the family where the forest’s pivot count per oracle is largest, which is not a coincidence of constants but a difference in what each method spends its work on.

9.2 The dual simplex

Table 15: Campaign dual: paired speedup against forest-primal on the instances both settings solved and verified.

configuration	paired	median	min	max
forest-dual	28	0.90	0.34	2.47
forest-dual-largest-tree	27	0.78	0.31	2.38
forest-primal	31	1.00	1.00	1.00
parametric	31	3.89	0.98	134.61

Table 14: Campaign dual: instances solved and verified, median and worst median wall time in seconds, PAR-2 with a 60s limit, and peak resident memory in MiB.

family	configuration	solved	median	worst	PAR-2	MiB
PCKP (historical)	forest-dual	1/1	0.098	0.098	0.10	23
PCKP (historical)	forest-dual-largest-tree	1/1	0.137	0.137	0.14	23
PCKP (historical)	forest-primal	1/1	0.053	0.053	0.05	23
PCKP (historical)	parametric	1/1	0.011	0.011	0.01	23
chains	forest-dual	3/3	0.239	0.253	0.24	26
chains	forest-dual-largest-tree	3/3	0.247	0.251	0.25	26
chains	forest-primal	3/3	0.080	0.099	0.08	26
chains	parametric	3/3	0.042	0.057	0.04	26
dense DAGs	forest-dual	0/3	–	–	120.00	53
dense DAGs	forest-dual-largest-tree	1/3	16.362	16.362	120.00	57
dense DAGs	forest-primal	3/3	11.117	15.094	11.12	57
dense DAGs	parametric	3/3	1.445	1.455	1.44	64
disconnected	forest-dual	3/3	1.594	3.239	1.59	44
disconnected	forest-dual-largest-tree	3/3	1.601	3.159	1.60	44
disconnected	forest-primal	3/3	1.100	1.259	1.10	44
disconnected	parametric	3/3	0.060	0.065	0.06	44
layered DAGs	forest-dual	3/3	7.379	7.712	7.38	44
layered DAGs	forest-dual-largest-tree	3/3	14.012	15.000	14.01	44
layered DAGs	forest-primal	3/3	6.726	6.976	6.73	44
layered DAGs	parametric	3/3	0.069	0.090	0.07	44
no arcs	forest-dual	3/3	0.004	0.005	0.00	22
no arcs	forest-dual-largest-tree	3/3	0.005	0.007	0.01	22
no arcs	forest-primal	3/3	0.005	0.006	0.01	22
no arcs	parametric	3/3	0.005	0.005	0.00	22
precedence forests	forest-dual	3/3	0.010	0.010	0.01	22
precedence forests	forest-dual-largest-tree	3/3	0.010	0.010	0.01	22
precedence forests	forest-primal	3/3	0.013	0.014	0.01	22
precedence forests	parametric	3/3	0.005	0.007	0.00	22
sparse DAGs	forest-dual	6/6	0.682	2.069	0.68	44
sparse DAGs	forest-dual-largest-tree	6/6	0.686	2.364	0.69	44
sparse DAGs	forest-primal	6/6	0.541	1.320	0.54	44
sparse DAGs	parametric	6/6	0.039	0.077	0.04	44
transitive arcs	forest-dual	3/3	0.011	0.022	0.01	23
transitive arcs	forest-dual-largest-tree	3/3	0.010	0.021	0.01	23
transitive arcs	forest-primal	3/3	0.019	0.023	0.02	23
transitive arcs	parametric	3/3	0.007	0.008	0.01	23
two levels	forest-dual	3/3	11.025	13.487	11.02	26
two levels	forest-dual-largest-tree	1/3	20.905	20.905	120.00	26
two levels	forest-primal	3/3	7.784	11.685	7.78	25
two levels	parametric	3/3	0.111	0.166	0.11	26

The measurements above say that the primal loop is dominated by zero-step pivots. Primal

degeneracy and dual degeneracy are different defects, and here they are wildly asymmetric: the primal kind is a per-pivot phenomenon, at a median of 99.3% of pivots over the whole dual campaign, while the only source of the dual kind is a tie between block ratios, which is a per-block event and therefore bounded by the number of blocks. That asymmetry is the standard indication for the dual method, and it was implemented and measured.

Two structural facts make it unusually clean on this problem. The vertex variables are never primal infeasible, since their value is $1/w(P)$ on the positive component and zero elsewhere, so every primal infeasibility is a precedence slack, that is an arc leaving P ; and the basis is primal feasible exactly when P is closed, which is therefore the stopping rule. The starting basis — every vertex its own tree, P the vertex of maximum ratio — is dual feasible by construction and costs $O(n)$.

The prediction is confirmed on the quantity it was made about, and refuted on every other. Table 14 and table 15 report the dedicated campaign on the frozen validation split, 31 instances in two replicas under the same protocol as every other campaign.

Degeneracy collapses: the primal sits at a median of 99.3% of pivots with zero step, the dual at 0.0%. Nothing else improves. The dual performs *more* pivots, not fewer — a median of 7 813 against the primal’s 3 253 on the instances both solve — and it is slower: the paired speedup against the primal has a median of 0.90, ranging from 0.34 to 2.47. It also solves less: 56 of 62 runs against 62 of 62, losing all three replicas of a dense DAG of two thousand vertices that the primal solves in about eleven seconds. The variant that picks the largest tree is worse again, median 0.78 and 54 of 62.

So the reduction of degeneracy was real and worth nothing. The dual traded null steps for a longer path, which is the outcome that the primal’s own diagnosis should have predicted: the cost was never the null steps as such, it was the number of pivots needed to walk to the optimum, and the dual walks farther. On the same instances the parametric backend is 3.89 times the primal at the median and up to 135 times on one of them.

Choosing the leaving arc by a structural criterion instead of by smallest index does not help either, and there is a structural reason: every violated slack has the same value $-1/w(P)$, so a most-infeasible rule has nothing to discriminate. The largest-tree rule is the one carried into the campaign, and it is the slowest of the three settings measured.

Correctness is not at stake in any of this. The dual returns the same canonical blocks as the primal on the whole exhaustive suite and on fifteen hundred random instances checked with the diagnostics enabled at every pivot, and it produces its certificate through the same verified code. If dual feasibility is lost to rounding, the solver restarts from the primal initialization rather than inheriting an infeasible basis, so the answer is certified either way.

10 Scalability and many capacities

10.1 One capacity against many

Table 17: Campaign capacities: instances solved and verified, median and worst median wall time in seconds, PAR-2 with a 60 s limit, and peak resident memory in MiB.

family	configuration	solved	median	worst	PAR-2	MiB
PCKP (historical)	forest-q1	1/1	0.051	0.051	0.05	33
PCKP (historical)	forest-q100	1/1	0.092	0.092	0.09	43
PCKP (historical)	forest-q20	1/1	0.061	0.061	0.06	33
PCKP (historical)	forest-q5	1/1	0.050	0.050	0.05	33
PCKP (historical)	parametric-q1	1/1	0.010	0.010	0.01	47

continua

continua dalla pagina precedente

family	configuration	solved	median	worst	PAR-2	MiB
PCKP (historical)	parametric-q100	1/1	0.058	0.058	0.06	47
chains	forest-q1	3/3	0.082	0.094	0.08	210
chains	forest-q100	3/3	0.158	0.167	0.16	210
chains	forest-q20	3/3	0.096	0.107	0.10	210
chains	forest-q5	3/3	0.083	0.097	0.08	210
chains	parametric-q1	3/3	0.041	0.052	0.04	210
chains	parametric-q100	3/3	0.124	0.136	0.12	210
dense DAGs	forest-q1	3/3	11.296	14.783	11.30	1072
dense DAGs	forest-q100	3/3	12.466	15.503	12.47	1072
dense DAGs	forest-q20	3/3	11.288	13.853	11.29	1072
dense DAGs	forest-q5	3/3	10.905	13.799	10.91	1072
dense DAGs	parametric-q1	3/3	1.440	1.500	1.44	1072
dense DAGs	parametric-q100	3/3	2.952	2.962	2.95	1073
disconnected	forest-q1	3/3	1.126	1.281	1.13	1073
disconnected	forest-q100	3/3	1.245	1.391	1.25	1073
disconnected	forest-q20	3/3	1.139	1.301	1.14	1073
disconnected	forest-q5	3/3	1.167	1.275	1.17	1073
disconnected	parametric-q1	3/3	0.067	0.080	0.07	1073
disconnected	parametric-q100	3/3	0.156	0.159	0.16	1073
layered DAGs	forest-q1	3/3	6.182	7.267	6.18	1073
layered DAGs	forest-q100	3/3	6.919	7.152	6.92	1073
layered DAGs	forest-q20	3/3	7.046	7.294	7.05	1073
layered DAGs	forest-q5	3/3	6.223	7.003	6.22	1073
layered DAGs	parametric-q1	3/3	0.071	0.098	0.07	1073
layered DAGs	parametric-q100	3/3	0.185	0.186	0.18	1073
no arcs	forest-q1	3/3	0.005	0.006	0.01	25
no arcs	forest-q100	3/3	0.014	0.014	0.01	25
no arcs	forest-q20	3/3	0.007	0.008	0.01	25
no arcs	forest-q5	3/3	0.006	0.006	0.01	25
no arcs	parametric-q1	3/3	0.005	0.005	0.00	25
no arcs	parametric-q100	3/3	0.012	0.013	0.01	25
precedence forests	forest-q1	3/3	0.008	0.011	0.01	26
precedence forests	forest-q100	3/3	0.019	0.021	0.02	26
precedence forests	forest-q20	3/3	0.011	0.012	0.01	26
precedence forests	forest-q5	3/3	0.010	0.011	0.01	26
precedence forests	parametric-q1	3/3	0.005	0.005	0.01	26
precedence forests	parametric-q100	3/3	0.016	0.016	0.02	26
sparse DAGs	forest-q1	6/6	0.542	1.366	0.54	1073
sparse DAGs	forest-q100	6/6	0.602	1.833	0.60	1073
sparse DAGs	forest-q20	6/6	0.564	1.496	0.56	1073
sparse DAGs	forest-q5	6/6	0.554	1.380	0.55	1073
sparse DAGs	parametric-q1	6/6	0.044	0.084	0.04	1073
sparse DAGs	parametric-q100	6/6	0.101	0.217	0.10	1073
transitive arcs	forest-q1	3/3	0.015	0.017	0.01	33
transitive arcs	forest-q100	3/3	0.036	0.040	0.04	33
transitive arcs	forest-q20	3/3	0.019	0.021	0.02	33
transitive arcs	forest-q5	3/3	0.015	0.017	0.02	33
transitive arcs	parametric-q1	3/3	0.006	0.007	0.01	33
transitive arcs	parametric-q100	3/3	0.029	0.034	0.03	33
two levels	forest-q1	3/3	7.173	12.069	7.17	178
two levels	forest-q100	3/3	7.505	11.827	7.50	209
two levels	forest-q20	3/3	6.837	11.506	6.84	178
two levels	forest-q5	3/3	6.827	11.531	6.83	178
two levels	parametric-q1	3/3	0.094	0.137	0.09	209

continua

continua dalla pagina precedente

family	configuration	solved	median	worst	PAR-2	MiB
two levels	parametric-q100	3/3	0.326	0.349	0.33	210

The canonical positive path is built once and answers any capacity: locating the critical block costs $O(\log k)$ in the number of blocks, but exporting a dense solution vector costs at least $O(n)$ per query. The campaign therefore separates the cost of the path from the cost of the queries, and reports the number of queries at which building the whole path pays for itself against solving each capacity independently.

11 Degeneracy, ties and numerics

11.1 Why the pivot loop stalls

The normalised subproblem is degenerate by construction. In a basis the vertex values are constant on the positive component and zero on every other tree, so a step is blocked at length zero whenever the blocking variable is one of the many already at zero: a vertex of the entering subtree with a negative direction component, or a precedence slack between two vertices that are both at zero. On the applied instances the median share of pivots with zero step is 99.3%, and it reaches 99.9%; on the family with no arcs at all, where no slack can block, the share is zero. Degeneracy is therefore a property of the precedence structure, not an accident of the implementation.

This observation drove three changes, each kept as a separate option so that section 9 can measure them one at a time.

The anti-cycling fallback must not be the operating policy. Switching permanently to Bland’s rule after a run of degenerate pivots made every entering rule collapse onto Bland within the first oracle, because the run of degenerate pivots never ends. Moving the trigger far away, and releasing the fallback at the first pivot with a positive step, restores the improving rule without giving up termination: an episode under Bland runs on a fixed basic solution and cannot cycle, and every episode ends with a strict increase of the ratio, so only finitely many episodes are possible.

The ratio test need not scan every arc. The direction is $+\sigma$ on the entering subtree, $-\kappa$ on the positive component and zero elsewhere, so an arc can block only if it crosses the boundary of one of the two sets. Because the positive component is closed, an arc that crosses it has its tail outside, which makes the restricted scan complete.

A zero step is the minimum, so the scan can stop at the first one. This is the shortcut the historical prototype implemented as an explicit feasibility check during the forest visit. It is suspended while Bland’s rule is active, because that rule needs the smallest index among ties.

11.2 Ties and numerics

Exact ties are handled by construction: the canonical block is the union of all maximisers, obtained either by complementarity from the dual or by aggregating consecutive increments of equal ratio. Both settings return the same blocks on the whole exact suite. Near-ties are separated by the relative optimality tolerance and are deliberately not merged.

All arithmetic is `long double` with binary scaling, which is exact in binary floating point and does not turn an integer zero-profit sum into a spurious nonzero ratio. Certificates are re-checked

Table 16: Campaign scalability: instances solved and verified, median and worst median wall time in seconds, PAR-2 with a 60s limit, and peak resident memory in MiB.

family	configuration	solved	median	worst	PAR-2	MiB
chains	dinkelbach	18/18	0.234	12.762	0.23	61
chains	forest-tuned	18/18	0.094	16.155	0.09	61
chains	parametric	18/18	0.050	2.570	0.05	61
diamonds	dinkelbach	9/9	0.239	11.757	0.24	61
diamonds	forest-tuned	9/9	0.112	14.689	0.11	61
diamonds	parametric	9/9	0.045	2.057	0.04	61
layered DAGs	dinkelbach	23/33	1.522	23.139	2.32	107
layered DAGs	forest-tuned	13/33	0.024	7.336	120.00	92
layered DAGs	parametric	33/33	0.080	3.340	0.08	112
no arcs	dinkelbach	16/18	0.230	4.270	0.25	62
no arcs	forest-tuned	18/18	0.057	8.700	0.06	62
no arcs	parametric	18/18	0.037	6.428	0.04	62
sparse DAGs	dinkelbach	18/24	0.706	8.630	0.89	62
sparse DAGs	forest-tuned	15/24	1.014	8.667	2.00	62
sparse DAGs	parametric	24/24	0.087	4.914	0.09	62

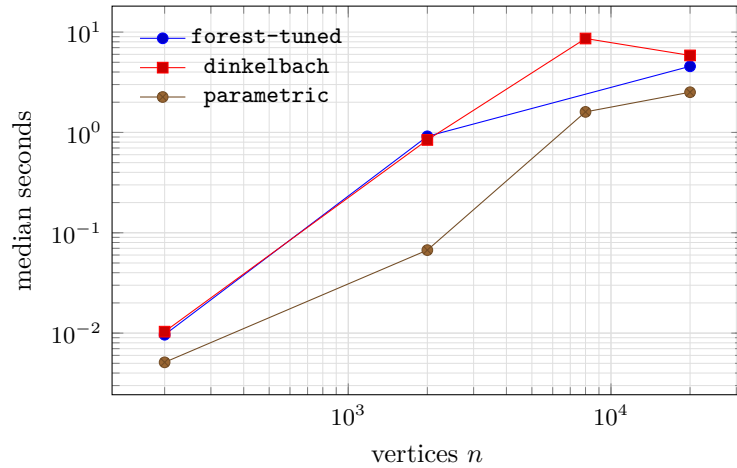


Figure 2: Median wall time against the number of vertices on the generated families whose structure is held fixed while the size varies. Only runs that finished and were verified appear; a configuration whose curve stops has hit the time limit beyond that size.

in the original units, after expanding strongly connected components, so a numerical success in the scaled model cannot pass as a success in the user’s model.

12 Discussion

12.1 What the measurements support

The three backends agree on every instance of the exact suite and on every instance of the campaigns they both solved: same canonical blocks, same LP values, each with its own certificate produced by code that shares nothing with the others. Together with the independent LP solver and the exhaustive rational oracle, this supports the correctness of the implementation on the domain it claims: generic DAGs, profits of either sign, strictly positive weights, one capacity, cyclic digraphs by condensation.

On performance the picture is uneven, and it is given per family because a single number would hide the shape of it. The parametric backend solves the whole test set; Dinkelbach solves 40 of 46, the tuned ratio-closure simplex 36 and the reference Ratio-closure simplex 27. The tuned ratio-closure simplex is not the fastest even where it finishes: its median over solved instances, 0.142 s, is above the parametric backend’s 0.057 s, and it is taken over the smaller set it can finish.

Per family, the tuned ratio-closure simplex finishes everything on chains, diamonds, strongly connected components, empty graphs, precedence forests, dense and sparse DAGs, and the historical precedence-knapsack set; the parametric backend matches or beats it on time in every one of those families — a tie on the trivial ones, a factor of 6 on the historical instances and a factor of 29 on the sparse DAGs. It solves 3 of 9 layered DAGs where the parametric backend solves all nine and Dinkelbach eight. On disconnected graphs and on the MineLib-derived instances it solves nothing within the limit, and neither does Dinkelbach; only the parametric backend does. The claim that the ratio-closure simplex and Dinkelbach are comparable on the applied instances, which an earlier reading of a handful of cases suggested, is not supported by the frozen test set: on the historical precedence-knapsack family both finish everything, and the medians are 0.087 s for Dinkelbach against 0.655 s.

One further correction belongs here rather than in a footnote. The ablation shows two settings that the parameter search rejected and that are faster than the frozen configuration on the paired comparison: the sink initial basis (3.33 against 2.83) and transitive reduction enabled (3.46). The search was run and frozen honestly, on training and validation data, but its choice is not the best one available in the space it searched, and a second search with a larger budget is the obvious next step.

12.2 What changed, and why it matters more than the parameters

The first measurements of this project had the ratio-closure simplex implementation one to three orders of magnitude behind both baselines, with time-outs on medium instances. The gap was not closed by tuning the pricing rule, which barely moves the result, but by three structural corrections identified from the degeneracy analysis of section 11: releasing the anti-cycling fallback instead of letting it become the operating policy, restricting the ratio-test scan to the arcs that can actually block, and stopping that scan at the first zero-step blocker. The last two reproduce techniques already present in the group’s historical prototype, which had no anti-cycling rule at all and carried an explicit feasibility shortcut for exactly this purpose.

The lesson recorded here is that in this problem class the entering rule is close to irrelevant while the treatment of degeneracy dominates. Parameter search over pricing schemes, block sizes and candidate limits produced differences within a few per cent; the degeneracy-related settings produced factors.

12.3 What the measurements do not support

They do not license any statement about the state of the art in maximum flow. The flow backends use a Dinic implementation written for this project; they do not use pseudoflow, highest-label push-relabel with gap and global relabelling, or the amortised flow reuse of a proper parametric algorithm. A comparison against a competitive flow code would very likely widen the gap on the families where ratio-closure simplex already loses, and could also narrow the cases where it currently wins.

They also do not support any claim of novelty. The audit of the literature has not been carried out in this phase, by explicit decision, and `docs/claims.md` keeps every novelty row in the *to be verified* state.

12.4 What follows

Two directions are supported by the measurements rather than by preference. First, pricing and forest rebuilding are the dominant costs of a pivot, together about four fifths of the solver's time on the hardest family, and both are recomputed in full where only a small part of the graph changed: incremental pricing and updates confined to the modified paths are the natural next step. Second, and more important, the length of the pivot path is what actually separates this method from the parametric backend, and neither the dual formulation nor the leaving rules touched it.

A third direction is now ruled out rather than recommended. The dual simplex was implemented and measured on a dedicated campaign precisely because the primal is so degenerate, and it does remove the degeneracy it was predicted to remove; but on the validation split it does not beat the primal, and the encouraging factors observed on three hard instances did not survive the larger sample. That is the reason the split exists.

13 Reproducibility

Every artefact in this report is derived from files stored in the project. `tools/report/analyze.py` first checks each campaign manifest for completeness — planned runs against stored runs, for the build the manifest describes — and refuses to emit tables for a campaign that fails the check. It then writes the tables and figure data used here, the per-run CSV files in `experiments/analysis/`, and `provenance.json` recording, for each campaign, the build fingerprint, the run counts, the status histogram and a hash of the run records.

A run is identified by instance content hash, task, capacity mode, resolved configuration, build source hash and replica index. A finished run is never overwritten and never recomputed: a different build or configuration produces a different identifier. Interrupting a campaign and restarting it resumes from the stored state.

The commands that rebuild the whole chain — environment, instance import and generation, frozen splits, campaigns, tuning, analysis and this document — are listed in `docs/reproducibility.md`. A reduced reproduction that repeats one significant comparison in a few minutes is available as `configs/suites/smoke.json`.

The historical material was read, never modified: archives were expanded into a temporary directory outside the project, and `provenance/original\_manifest.sha256` records size and SHA-256 of every original file used, in a form that `sha256sum -check` accepts.

A Complete tables and campaign details

The pilot campaign fixed the sizes, the time limit and the number of replicas of the later campaigns. It also exposed a protocol defect that was corrected before any measurement was kept: the time limit was enforced by the ratio-closure simplex pivot loop but not by the flow

Table 18: Pilot campaign: median wall time in seconds of one solve per instance and backend. A status word replaces the time when the run did not finish within the pilot limit.

instance	n	m	dinkelbach	forest-reference	forest-tuned	parametric
pckp_L_349_2101	349	2101	0.008	0.212	0.030	0.007
dense_density0.1_n800_neg30_r0	800	31979	0.652	time_limit	0.597	0.120
pckp_A_972_1661	972	1661	0.098	0.489	0.074	0.019
minelib_newman1	1060	3922	0.187	6.292	1.015	0.023
chain_plain_n2000_neg30_r0	2000	1999	0.231	0.663	0.074	0.048
cyclic_components10_n2000_neg30_r0	2000	2009	0.007	0.008	0.007	0.007
disconnected_components6_n2000_ne	2000	1993	0.906	4.365	1.063	0.062
empty_plain_n2000_neg30_r0	2000	0	0.243	0.337	0.055	0.033
layered_levels8-density0.05_n2000	2000	21972	2.405	time_limit	time_limit	0.100
sparse_degree3_n2000_neg30_r0	2000	5976	0.686	time_limit	1.024	0.061
transitive_degree2-extra3.0_n2000	2000	15972	1.528	time_limit	1.847	0.114
sparse_degree3_n20000_neg30_r0	20000	59964	time_limit	time_limit	time_limit	3.313

backends, so a single run had exceeded the budget. The limit is now checked before every maximum flow, and the pilot was rerun from scratch with the corrected build. Since a run’s identity includes the build hash, the earlier results could not be silently reused.

Per-run records, including failed and timed-out runs, are in `experiments/analysis/*.csv`; the campaign manifests in `experiments/runs/*/manifest.json` additionally store the resolved configuration of every run, its statistics counters and the outcome of its verification. The completeness report that gates the generation of every table in this document is `experiments/analysis/completeness.json`.

References

Valerio Dose, Fabio Furini, and Marco Locatelli. On parametric maximum closure problems over precedence forests. Manuscript, arXiv v1, 2026. Companion work: the same canonical sequence of closure layers on directed precedence forests. Code: <https://github.com/fabiofurini/parametric-closure-forests>.